

FGV EMap

João Pedro Jerônimo e Eduardo Adame

Aprendizado de Máquina

Revisão para A2

Rio de Janeiro

2026

Conteúdo

1	Problema de Aprendizagem	3
1.1	Introdução	4
1.2	Dilema viés-variância	4
2	Diferenciação Automática	5
2.1	Introdução	6
2.2	Diferenciação Automática forward-mode	6
2.3	Diferenciação Automática reverse-mode	8
2.4	Exemplo em código	9
3	Processos Gaussianos	12
3.1	Revisitando a priori Gaussiana	13
3.1.1	Funções de kernel comuns	13
3.1.2	Construindo funções de kernel	14
3.2	GPs para regressão	14
3.2.1	Escolhendo os parâmetros do kernel	15
3.3	GPs para classificação	16
4	Graph Neural Networks	17
4.1	Introdução	18
4.2	Notações	18
4.3	Usos de GNNs	18
4.4	Passagem de Mensagem	19
4.5	Graph Convolutional Network (GCN)	20
5	Convolutional Neural Networks (CNN)	21
5.1	Introdução	22
5.2	Imagens como dados	22
5.3	Filtros	22
5.3.1	Capturando Localidade	23
5.4	Equivariância em Translação	24
5.5	Padding	24
5.6	Convoluções com Stride	25
5.7	Convolução Multidimensional	25
5.8	Pooling	26
5.9	Arquiteturas	27
5.10	Os Gradientes	27
5.10.1	Convolução sem Padding e Stride	27
5.10.2	Convolução com Padding e Stride	30
5.11	Otimizações Computacionais	33
5.11.1	im2col	34
5.11.2	col2im	36
5.11.3	Aplicação como uma Multiplicação de Matrizes	36
5.12	Pooling	36
	Referências	38

Problema de Aprendizagem

1.1 Introdução

Em aprendizado supervisionado, o objetivo principal é criar uma aproximação $h : \mathcal{X} \rightarrow \mathcal{Y}$ para uma função $f : \mathcal{X} \rightarrow \mathcal{Y}$ a partir de amostras $D = \{(x_n, y_n)\}_{n=1}^N$, onde cada exemplo de treinamento é uma amostra independente proveniente de $\mathbb{P}_{x,y}$ e y_n é uma observação (possivelmente ruidosa) de $f(x_n)$. No caso de regressão, poderíamos usar vários métodos distintos para construir h . Alguns exemplos que já vimos são k -NN, regressão linear e redes neurais RBF. Além disso, podemos alterar drasticamente o comportamento desses modelos alterando hiper-parâmetros. Isso nos leva às duas perguntas estruturantes desse capítulo:

1. Qual é a melhor classe de modelos (e.g., regressão linear, rede RBF) para cada problema?
2. Como escolher os melhores hiper-parâmetros para cada classe de modelos?

1.2 Dilema viés-variância

De maneira geral, aprendemos h usando o conjunto de treino D com N amostras (i.e., $|D| = N$), mas queremos que h apresente boa performance em novas observações de $x \sim \mathbb{P}_x$. Em outras palavras, gostaríamos de um modelo que minimiza uma função de perda ℓ média $\mathbb{E}_x[\ell(h(x), f(x))]$. Esse valor é comumente chamado de erro de generalização.

Para a seguinte discussão, assuma que ℓ é o erro quadrático — i.e., $\ell(y, \hat{y}) = (y - \hat{y})^2$. Portanto o erro de generalização é dado por $\mathbb{E}_x[(h(x) - f(x) - \varepsilon)^2]$ (ε é um ruído de **média 0**). Como h é o resultado de um processo de aprendizado, ele depende diretamente dos exemplos em D . Por exemplo, se estamos fazendo regressão linear, h pode ser obtida aplicando via máxima verossimilhança. No entanto, podemos analisar h de maneira mais geral, sem se prender aos valores específicos em D . Para tal, calculamos o valor esperado do erro tratando D como uma variável aleatória. Isto é, calculamos uma média ponderada sobre todos os possíveis bancos de dado de tamanho N — com pesos dados por $\mathbb{P}_{x,y}$. Usando a linearidade do operador de esperança e a independência de D e x , segue que:

$$\begin{aligned}\mathbb{E}_{x,D,\varepsilon}[(h_D(x) - f(x) - \varepsilon)^2] &= \mathbb{E}_x\left[\mathbb{E}_{D,\varepsilon}\left[(h_{D(x)} - f(x) - \varepsilon)^2\right]\right] \\ &= \mathbb{E}_x\left[\mathbb{E}_D[(h_D(x) - f(x))^2] + \mathbb{E}_\varepsilon[\varepsilon^2]\right] \\ &= \mathbb{E}_x[\mathbb{V}_D[h_D(x)]] + \mathbb{E}_x\left[(\mathbb{E}_D[h_D(x)] - f(x))^2\right] + \mathbb{E}_\varepsilon[\varepsilon^2]\end{aligned}\tag{1}$$

Como isso muda nossa vida? Na maioria das aplicações, temos pouca informação sobre f . Nesse caso, nosso primeiro instinto talvez fosse escolher um método capaz de gerar aproximações arbitrariamente intrincadas. No entanto, métodos mais flexíveis costumam estar associados à uma maior variância no processo de aprendizado, o que influencia negativamente a performance esperada para novos dados de entrada. Por outro lado, métodos simples como regressão linear possuem baixa variância, mas podem apresentar alto viés caso f não possa ser bem aproximada por um (hiper-)plano. Em suma, não existe uma bala de prata. Precisamos de protocolos empíricos bem definidos para escolher o método mais adequado para cada situação.

Diferenciação Automática

2.1 Introdução

Aposto que, se você é alguém que, como eu, sempre teve interesse de ver esses algoritmos de machine learning e implementar eles do 0, sentiu alguma dificuldade principalmente quando a sua implementação envolvia o cálculo de algum gradiente (principalmente algum gradiente difícil). Calcular o gradiente de uma função $g : \mathbb{R}^D \rightarrow \mathbb{R}$ com relação a $x \in \mathbb{R}^D$ é uma tarefa que pode ser extremamente trabalhosa, especialmente quando g é uma função complexa, e justamente essa tarefa complexa é a base fundamental pra que nossas máquinas possam aprender.

Existem, essencialmente, 4 formas de se calcular o gradiente de uma rede neural.

1. Derivar as expressões analiticamente e implementá-las manualmente via software. Essa abordagem é extremamente trabalhosa, propensa a erros e não escalável para funções complexas. (Particularmente, exatamente o que eu sempre tentava fazer). Não só isso, mas as implementações dessas alternativas envolvem montar funções distintas para o forward pass e o backward pass, o que torna o processo ainda mais trabalhoso.
2. Outro método é calcular o gradiente de forma aproximada:

$$\frac{\partial f}{\partial x} \approx \frac{f(x+h) - f(x)}{h} \quad h \approx 0 \quad (2)$$

ele está bastante sujeito à erros numéricos, mas esse não é o principal problema, mas sim que ele escala muito mal com o tamanho da rede neural, mas ele é muito útil para verificar se a implementação do gradiente está correta (já que ele utiliza apenas o forward pass da rede neural).

3. O terceiro método é o **symbolic differentiation**, que consiste em derivar a função simbolicamente e depois implementá-la. Essa abordagem é mais escalável do que a primeira, mas ainda assim pode ser trabalhosa e propensa a erros, especialmente para funções complexas. Por exemplo, considere uma função $f(x) = u(x) \cdot v(x)$,

$$\frac{\partial f}{\partial x} = \frac{\partial u}{\partial x} \cdot v + u \cdot \frac{\partial v}{\partial x} \quad (3)$$

e se u e v forem funções complexas, a derivada de f pode se tornar extremamente complicada. Além disso, o symbolic differentiation pode gerar expressões redundantes, o que pode levar a uma implementação ineficiente.

4. O quarto e último método é o **automatic differentiation**, que é o método mais eficiente e escalável para calcular gradientes de funções complexas. Ele é baseado na regra da cadeia e na decomposição da função em operações elementares, permitindo calcular o gradiente de forma eficiente e precisa. Além disso, ele permite calcular o gradiente de funções complexas sem a necessidade de derivar manualmente as expressões analiticamente.

2.2 Diferenciação Automática forward-mode

Considere a seguinte função:

$$f(x_1, x_2) = x_1 x_2 + e^{x_1 x_2} - \sin(x_2) \quad (4)$$

quando implementado em código, podemos decompor a função em operações elementares que podem ser visualizadas em grafo

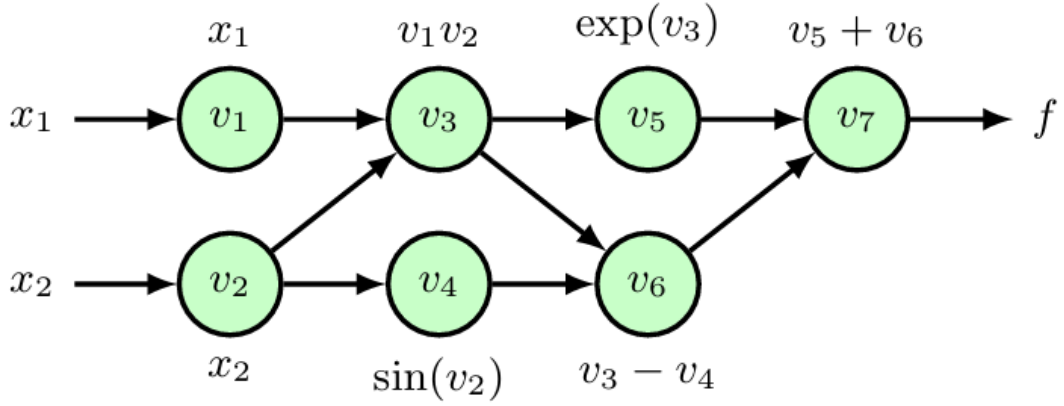


Figura 1: Grafo de computação da função (4)

essas operações são chamadas de *evaluation trace*

$$\begin{aligned}
 v_1 &= x_1 \\
 v_2 &= x_2 \\
 v_3 &= x_1 x_2 \\
 v_4 &= \sin(x_2) \\
 v_5 &= e^{v_3} \\
 v_6 &= v_3 - v_4 \\
 v_7 &= v_5 + v_6
 \end{aligned} \tag{5}$$

Agora suponha que precisamos calcular o gradiente $\frac{\partial f}{\partial x_1}$. Nós definimos a **variável tangente** como $\dot{v}_i = \frac{\partial v_i}{\partial x_1}$. Podemos calcular essa variável automaticamente com a regra da cadeia:

$$\dot{v}_i = \frac{\partial v_i}{\partial x_1} = \sum_{j \in \text{pa}(v_i)} \frac{\partial v_i}{\partial v_j} \cdot \frac{\partial v_j}{\partial x_1} = \sum_{j \in \text{pa}(v_i)} \frac{\partial v_i}{\partial v_j} \cdot \dot{v}_j \tag{6}$$

onde $\text{pa}(v_i)$ é o conjunto de pais de v_i no grafo de computação. Por exemplo, para $v_3 = x_1 x_2$, temos que $\text{pa}(v_3) = \{v_1, v_2\}$. Resolvendo de acordo com a equação acima, obtemos os valores de $\dot{v}_i$ para cada v_i :

$$\begin{aligned}
 \dot{v}_1 &= 1 \\
 \dot{v}_2 &= 0 \\
 \dot{v}_3 &= v_1 \dot{v}_2 + v_2 \dot{v}_1 \\
 \dot{v}_4 &= \cos(v_2) \dot{v}_2 \\
 \dot{v}_5 &= e^{v_3} \dot{v}_3 \\
 \dot{v}_6 &= \dot{v}_3 - \dot{v}_4 \\
 \dot{v}_7 &= \dot{v}_5 + \dot{v}_6
 \end{aligned} \tag{7}$$

Podemos resumir a **diferenciação automática** para este exemplo da seguinte forma:

Primeiro, escrevemos um código para implementar a avaliação das **variáveis primais (primal variables)**, dadas pelas equações (8.50) a (8.56). As equações associadas e o código correspondente para avaliar as **variáveis tangentes (tangent variables)**, dadas pelas equações (8.58) a (8.64), são gerados automaticamente.

Para calcular a derivada $(\frac{\partial f}{\partial x_1})$, fornecemos valores específicos para (x_1) e (x_2) , e então o código executa as equações primais e tangentes, avaliando numericamente, em sequência, os pares $((v_i, \dot{v}_i))$ até obtermos $((\dot{v}_5))$, que é a derivada desejada.

Agora considere o cenário onde temos uma função vetorial, onde a segunda saída é dada por:

$$f_2(x_1, x_2) = (x_1 x_2 - \sin(x_2)) \exp(x_1 x_2) \quad (8)$$

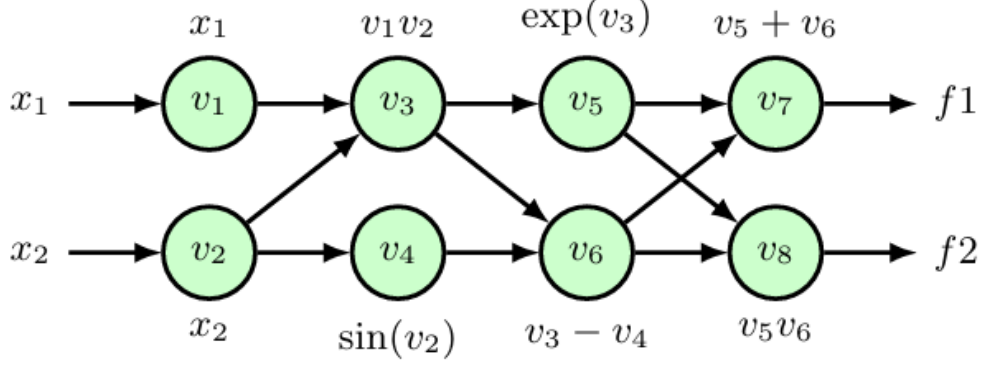


Figura 2: Grafo de computação da função (4) e (8)

Se quisermos calcular $\frac{\partial f_2}{\partial x_1}$, conseguimos fazer isso no mesmo forward pass do cálculo de $\frac{\partial f_1}{\partial x_1}$, apenas adicionando mais equações para as **variáveis primais** e **variáveis tangentes** correspondentes a f_2 . No entanto, se quisermos calcular $\frac{\partial f_1}{\partial x_2}$, precisamos fazer um novo forward pass, pois a variável tangente $\dot{v}_i$ depende da variável de entrada que estamos diferenciando. Portanto, no geral, se temos uma função com D inputs e K outputs, então um único forward pass produz apenas uma coluna da matriz jacobiana $K \times D$

$$J = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_D} \\ \vdots & \vdots & \vdots \\ \frac{\partial f_K}{\partial x_1} & \dots & \frac{\partial f_K}{\partial x_D} \end{pmatrix} \quad (9)$$

A diferenciação automática forward-mode é muito útil principalmente em casos onde temos muito mais outputs do que inputs ($K \gg D$). Entretanto, no contexto de machine learning, o mais comum é termos uma única função de erro $\ell : \mathbb{R}^D \rightarrow \mathbb{R}$ que queremos minimizar com relação a milhões de parâmetros em cadeia dentro das redes neurais, então a implementação forward-mode fica ineficiente, para contornar isso, mudamos para outra abordagem

2.3 Diferenciação Automática reverse-mode

Podemos pensar nessa implementação como uma generalização do processo de backpropagation. No modo forward, nós alimentamos cada variável intermediária v_i com variáveis adicionais, nesses casos chamadas de **variáveis adjuntas**, denotadas por $\bar{v}_i$. Considere novamente uma função de apenas 1 output na forma $f : \mathbb{R}^D \rightarrow \mathbb{R}$. A variável adjunta $\bar{v}_i$ é definida como:

$$\bar{v}_i = \frac{\partial f}{\partial v_i} \quad (10)$$

Podemos calcular esse valor automaticamente com a regra da cadeia:

$$\bar{v}_i = \frac{\partial f}{\partial v_i} = \sum_{j \in \text{ch}(v_i)} \frac{\partial f}{\partial v_j} \cdot \frac{\partial v_j}{\partial v_i} = \sum_{j \in \text{ch}(v_i)} \bar{v}_j \cdot \frac{\partial v_j}{\partial v_i} \quad (11)$$

onde $\text{ch}(v_i)$ representa o conjunto de variáveis que dependem de v_i (filhos do nó v_i). Considerando novamente o exemplo da função (4), podemos calcular as variáveis adjuntas $\bar{v}_i$ para cada v_i :

$$\begin{aligned}\bar{v}_7 &= 1 \\ \bar{v}_6 &= \bar{v}_7 \\ \bar{v}_5 &= \bar{v}_7 \\ \bar{v}_4 &= -\bar{v}_6 \\ \bar{v}_3 &= \bar{v}_5 v_5 + \bar{v}_6 \\ \bar{v}_2 &= \bar{v}_3 v_1 + \bar{v}_4 \cos(v_2) \\ \bar{v}_1 &= \bar{v}_3 v_2\end{aligned}\tag{12}$$

Observe que essas equações começam na saída (output) e fluem para trás através do grafo até as entradas (inputs). Mesmo com múltiplas entradas, apenas um único backward pass é necessário para calcular as derivadas.

Para uma função de erro de rede neural, as derivadas de E em relação aos pesos e vieses (biases) são obtidas como as variáveis adjuntas (adjoint variables) correspondentes. Entretanto, se tivermos mais de uma saída, será necessário executar um backward pass separado para cada variável de saída.

O reverse mode costuma exigir mais memória do que o forward mode, porque todas as variáveis primais intermediárias (intermediate primal variables) precisam ser armazenadas para que estejam disponíveis quando for necessário calcular as variáveis adjuntas durante a passagem para trás.

Em contraste, no forward mode, as variáveis primais e tangentes são calculadas conjuntamente durante o forward pass, de modo que as variáveis podem ser descartadas assim que forem utilizadas.

Por isso, em geral, o forward mode também é mais simples de implementar do que o reverse mode.

2.4 Exemplo em código

```
1  class SinLayer:
2      def forward(self, x):
3          self.x = x
4          return np.sin(x)
5
6      def backward(self, dout):
7          dx = dout * np.cos(self.x)
8          return dx
9
10
11 class SquaredLayer:
12     def forward(self, x):
13         self.x = x
14         return x ** 2
15
16     def backward(self, dout):
17         dx = dout * 2 * self.x
18         return dx
```

```
19
20
21 class ModuleList:
22     def __init__(self, layers = None):
23         self.layers = layers or []
24
25     def forward(self, x):
26         for layer in self.layers:
27             x = layer.forward(x)
28         return x
29
30     def backward(self, dout):
31         for layer in reversed(self.layers):
32             dout = layer.backward(dout)
33         return dout
34
35 import numpy as np
36
37 class SinLayer:
38     def forward(self, x):
39         self.x = x
40         return np.sin(x)
41
42     def backward(self, dout):
43         return dout * np.cos(self.x)
44
45
46 class SquaredLayer:
47     def forward(self, x):
48         self.x = x
49         return x ** 2
50
51     def backward(self, dout):
52         return dout * 2 * self.x
53
54
55 class ModuleList:
56     def __init__(self, layers=None):
57         self.layers = layers or []
58
59     def forward(self, x):
60         for layer in self.layers:
61             x = layer.forward(x)
62         return x
63
64     def backward(self, dout=1):
```

```
65         for layer in reversed(self.layers):
66             dout = layer.backward(dout)
67         return dout
68
69
70 # Função f representando sin^2(x)
71 f = ModuleList([
72     SinLayer(),
73     SquaredLayer()
74 ])
75
76 x = 2
77
78 y = f.forward(x)
79 dy = f.backward()
80
81 print(y)    # 0.8268218104...
82 print(dy)   # -0.7568024953...
```

Processos Gaussianos

3.1 Revisitando a priori Gaussiana

Como discutimos anteriormente, priors gaussianas são extremamente populares em modelos Bayesianos para regressão. Uma escolha comum, por exemplo, é colocar uma priori isotrópica $N(0, cI)$ sobre o vetor de pesos θ . Nesse caso, a priori sobre θ também induz implicitamente uma priori sobre $f(X') = X'\theta$ para qualquer $X' \in \mathbb{R}^{N' \times (D+1)}$ e $N' \in \mathbb{N}^+$. Mais especificamente, como $f(X')$ é uma transformação linear de variáveis Gaussianas, essa priori é Gaussiana com vetor de médias $\mu(X')$ e matriz de covariância $\Sigma(X', X')$ dados por:

$$\begin{aligned}\mu(X') &= \mathbb{E}_\theta[X'\theta] = X'\mathbb{E}_\theta[\theta] = 0 \\ \Sigma(X', X') &= \mathbb{E}_\theta[(X'\theta - 0)(X'\theta - 0)^T] = cX'X'^T\end{aligned}\tag{13}$$

Com essas observações em mente, podemos abstrair θ totalmente do nosso processo de aprendizado usando a seguinte priori sobre os valores de f :

$$f(X') \sim N(0, \Sigma(X', X')) \quad \forall X' \in \mathbb{R}^{N' \times (D+1)}, \quad N' \in \mathbb{N}^+\tag{14}$$

que é uma instância específica de um processo estocástico conhecido como **processo Gaussiano** (Gaussian process, GP). De forma geral, $(f(x))_{x \in \mathcal{X}}$ define um processo Gaussiano se qualquer vetor $[f(x_1), \dots, f(x_N)]^T$ com $x_1, \dots, x_N \in \mathcal{X}$ segue uma distribuição normal multivariada.

Definição 3.1.1 (Processo Gaussiano): Seja $\mathcal{X}$ um espaço de entradas. Dizemos que $(f(x))_{x \in \mathcal{X}}$ é um processo Gaussiano se, para qualquer conjunto finito de pontos $x_1, \dots, x_N \in \mathcal{X}$, o vetor $f = [f(x_1), \dots, f(x_N)]^T$ segue uma distribuição normal multivariada.

É importante ressaltar que a matriz de covariância $\Sigma(X', X')$ é proporcional à matriz Gramiana K (de produtos internos) dos vetores linha de X' , i.e., $K_{ij} = x'_i \cdot x'_j$, onde x'_i e x'_j denotam os vetores nas linhas i e j de X' , respectivamente. Além disso, incorporar uma função de expansão de base Φ no modelo da Equação (14) apenas implica em redefinir as entradas de K como $K_{ij} = k(x'_i, x'_j) = \Phi(x'_i) \cdot \Phi(x'_j)$. Em outras palavras, nosso GP sobre f pode ser completamente caracterizado por uma função de produto interno generalizada k — também conhecida como **função de kernel**. Por simplicidade notacional, denotaremos que f segue uma priori de GP como $f \sim \text{GP}(0, k)$. Nesse capítulo, assumiremos que a média de f é zero a priori; no entanto, seria possível utilizar uma função de média arbitrária $\mu(\cdot)$ com poucas alterações nos nossos desenvolvimentos.

Do ponto de vista de interpretação, funções de kernel nos permitem diretamente expressar como regularidades no espaço de entrada devem ser refletidas no espaço de saída. Além disso, existem casos em que Φ é computacionalmente intratável, mas seu kernel correspondente tem forma simples. Por exemplo, o kernel exponencial quadrático (ou Gaussiano), dado por $k(x, x') = \exp\left\{-\|x - x'\|_2^2 / 2\gamma^2\right\}$ é gerado a partir de uma expansão de base “infinita” — veja a Seção 4.2 do livro texto de Rasmussen e Williams (2006) para mais detalhes.

3.1.1 Funções de kernel comuns

Na literatura de GPs, existe uma variedade de funções de kernel criadas para modelar fenômenos distintos. No entanto, alguns kernels são extremamente populares, como:

1. Exponencial quadrático (ou Gaussiano):

$$k(x, x') = e^{-\|x - x'\|_2^2 / 2\gamma^2}\tag{15}$$

onde γ^2 é um hiperparâmetro que controla a suavidade da função de kernel;

2. Racional quadrático:

$$k(x, x') = \sigma^2 \left(1 + \frac{\|x - x'\|_2^2}{2\alpha\gamma^2} \right)^{-\alpha} \quad (16)$$

que corresponde a uma soma ponderada de kernels exponenciais quadráticos com diferentes larguras de banda;

3. Periódico:

$$k(x, x') = e^{-2\gamma^{-2} \sin^2\left(\pi \|x - x'\|_p \frac{1}{p}\right)} \quad (17)$$

que tem natureza periódica, com período p .

3.1.2 Construindo funções de kernel

A família das funções de kernel é fechada por um número de operações. Isto é, é possível manipular um kernel k de várias maneiras diferentes e ainda assim obter um kernel válido. Isso é extremamente útil em cenários em que precisamos, e.g., combinar propriedades de diferentes funções de kernel para refletir um fenômeno. Por exemplo, as seguintes operações resultam em kernels válidos:

1. Multiplicação por constante $c > 0$: $k'(x, x') = ck(x, x')$;
2. Produto: $k'(x, x') = k_1(x, x')k_2(x, x')$;
3. Soma: $k'(x, x') = k_1(x, x') + k_2(x, x')$;
4. Exponenciação: $k'(x, x') = e^{k_1(x, x')}$;
5. Multiplicação por função escalar avaliada em x e x' : $k'(x, x') = f(x)f(x')k(x, x')$.

3.2 GPs para regressão

Em problemas de regressão, é comum que a variável de resposta y_n para a entrada x_n seja observada com algum ruído. Seguindo o capítulo de regressão linear, suponha que $y = g(x) = f(x) + \varepsilon$ com $\varepsilon \sim N(0, \sigma^2)$. Note que isso é equivalente a dizer que $y|x \sim N(f(x), \sigma^2)$; portanto, podemos descrever um modelo qualquer de regressão com priori GP como:

$$\begin{aligned} y|f, x &\sim N(f(x), \sigma^2) \\ f &\sim \text{GP}(0, k) \end{aligned} \quad (18)$$

Como é de costume, estamos interessados em usar o modelo acima e o conjunto de treino $D = \{(x_n, y_n)\}_{n=1}^N$ para fazer previsões para, e.g., a variável de resposta y^* relativa a um novo vetor de atributos x^* . Em outras palavras, queremos avaliar $p(y^*|x^*, x_1, y_1, \dots, x_N, y_N)$. No entanto, existe um detalhe no modelo acima que facilitará nossa vida: a nossa combinação de priori e verossimilhança induz um processo Gaussiano sobre g . Mais especificamente, para qualquer $X' \in \mathbb{R}^{M \times D}$, $g(X')$ pode ser descrito como uma soma de duas variáveis aleatórias Gaussianas: $f(X')$ e um vetor M -dimensional ε com distribuição $N(0, \sigma^2 I)$. Portanto, temos que $g(X') \sim N(0, k(X', X') + \sigma^2 I)$.

Tomando $X' = \begin{pmatrix} x_1^T \\ \vdots \\ x_N^T \\ (x^*)^T \end{pmatrix}$, segue que:

$$\begin{pmatrix} y^* \\ y_1 \\ \vdots \\ y_N \end{pmatrix} \sim N \left(\begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} k(x^*, x^*) + \sigma^2 & k(x^*, x_1) & \dots & k(x^*, x_N) \\ k(x_1, x^*) & k(x_1, x_1) + \sigma^2 & \dots & k(x_1, x_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_N, x^*) & k(x_N, x_1) & \dots & k(x_N, x_N) + \sigma^2 \end{pmatrix} \right) \quad (19)$$

onde o lado direito está implicitamente condicionado nas entradas $x_1, \dots, x_N$ e x^* . Portanto, podemos obter a distribuição $p(y^* | x^*, x_1, y_1, \dots, x_N, y_N)$ simplesmente condicionando a Gaussiana acima nos valores observados $y_1, \dots, y_N$, i.e., $p(y^* | x^*, x_1, y_1, \dots, x_N, y_N) = N(\mu^*, S^*)$ com parâmetros μ^*, S^* dados por:

$$\begin{aligned}\mu^* &= k(x^*, X)(k(X, X) + \sigma^2 I)^{-1}y \\ S^* &= k(x^*, x^*) + \sigma^2 - k(x^*, X)(k(X, X) + \sigma^2 I)^{-1}k(x^*, X)^T\end{aligned}\quad (20)$$

onde X é a matriz com dados de treinamento e y é o vetor de suas respectivas respostas.

Interpretação de μ^* : Uma observação interessante é que μ^* é, essencialmente, uma combinação linear das respostas em y . Finalmente, vale ressaltar que σ^2 é comumente tratado como um hiperparâmetro, que podemos escolher com auxílio de um conjunto de validação.

3.2.1 Escolhendo os parâmetros do kernel

Já sabemos como construir um GP simples para regressão. No entanto, não discutimos como escolher os parâmetros da nossa função de kernel (e.g., a largura de banda γ do kernel Gaussiano). Como fazer isso? Na literatura de GPs, a saída comum é maximizar a esperança da verossimilhança sob a nossa priori, i.e., $p(y_1, \dots, y_N | x_1, \dots, x_N) = \mathbb{E}_{f \sim \text{GP}(0, k)}[N(y | f(X), \sigma^2 I)]$. Mais concretamente, denotando de maneira genérica os parâmetros da função de kernel por ω , os parâmetros ótimos podem ser calculados como:

$$\begin{aligned}\hat{\omega} &= \operatorname{argmax}_{\omega \in \Omega} \log p(y_1, \dots, y_N | x_1, \dots, x_N) \\ &= \operatorname{argmax}_{\omega \in \Omega} \log N(y | 0, k(X, X) + \sigma^2 I) \\ &= \operatorname{argmax}_{\omega \in \Omega} \left\{ -\frac{1}{2} \log \det(k(X, X) + \sigma^2 I) - \frac{1}{2} y^T (k(X, X) + \sigma^2 I)^{-1} y \right\}\end{aligned}\quad (21)$$

O procedimento descrito acima é comumente conhecido como **type II maximum likelihood**, **maximização da verossimilhança marginal** e **maximização da evidência** (note que estamos maximizando o denominador da regra de Bayes). Vale ressaltar que, ao contrário de MLE para regressão linear e logística, o problema acima costuma não ser convexo em ω . Por isso, é ideal conduzir otimização multi-start — e.g., rodando SGD usando diferentes pontos iniciais e escolhendo o melhor ótimo local obtido.

Estimando σ^2 : Note também que o procedimento que descrevemos assume que o ruído observacional σ^2 é uma constante. Nesse caso, poderíamos estimar os parâmetros do kernel para cada valor de σ^2 usando o conjunto de treino e escolher o σ^2 que resulta na maior evidência. Outra opção é otimizar σ^2 juntamente com ω , i.e.:

$$\hat{\sigma}^2, \hat{\omega} = \operatorname{argmax}_{\omega \in \Omega, \sigma^2 \in \mathbb{R}} \left\{ -\frac{1}{2} \log \det(k(X, X) + \sigma^2 I) - \frac{1}{2} y^T (k(X, X) + \sigma^2 I)^{-1} y \right\} \quad (22)$$

No entanto, otimizar σ^2 e ω juntos pode tornar nosso processo de aprendizado instável. Para aliviar esse problema, a comunidade de GPs em ML costuma usar algumas heurísticas de inicialização. A heurística mais comum consiste em fixar (a priori) a razão r^2 entre a variância da verossimilhança e da priori. Assumindo que nosso kernel é da forma $k = c^2 k'$, o primeiro passo é inicializar c com a variância das saídas de treinamento $y_1, \dots, y_N$. Subsequentemente, inicializamos o ruído observacional σ^2 com $\frac{c^2}{r^2}$. Nesse contexto, repetimos o processo de

otimização para diferentes valores de r^2 e escolhemos o resultado que leva ao melhor ótimo local.

3.3 GPs para classificação

Para classificações, vamos reformular como as saídas se comportam. Dessa vez, assumimos que

$$y_i \mid f_i, x_i \sim \text{Bernoulli}(\sigma(f_i(x_i))) \quad (23)$$

para simplificar notação, vou definir $\sigma_i = \sigma(f_i(x_i))$. Nós vamos aproximar $p(y|f, X)$ usando a aproximação de Laplace ou métodos de amostragem e depois achar a preditiva posteriori $p(y^*|X, y, x^*)$. Primeiro vamos achar a forma da verossimilhança

$$p(y_n|f_n, x_n) = \sigma_n^{y_n} (1 - \sigma_n)^{1-y_n} \Rightarrow p(y|f, X) = \prod_{n=1}^N \sigma_n^{y_n} (1 - \sigma_n)^{1-y_n} \quad (24)$$

agora, vamos escrever a posteriori de f dado X e y usando Bayes:

$$p(f|X, y) \propto p(y|f, X)p(f|X) \quad (25)$$

escrevendo em forma de log para facilitar as contas

$$\begin{aligned} \ln p(f|X, y) &= \ln p(y|f, X) + \ln p(f|X) + C \\ &= \sum_{n=1}^N \{y_n \ln \sigma_n + (1 - y_n) \ln(1 - \sigma_n)\} - \frac{1}{2} f(x)^T K^{-1} f(x) + C \end{aligned} \quad (26)$$

Agora podemos derivar para conseguir achar a moda m

$$\begin{aligned} \frac{\partial}{\partial f_n} \ln p(f|X, y) &= y_n(1 - \sigma_n) - (1 - y_n)\sigma_n - (K^{-1}f(x))_n = 0 \\ \Rightarrow \nabla \ln p(f|X, y) &= y - \sigma(f(x)) - K^{-1}f(x) = 0 \end{aligned} \quad (27)$$

essa equação não tem solução analítica, então utilizamos de métodos numéricos para achar a moda m que satisfaz

$$y - \sigma(m) - K^{-1}m = 0 \quad (28)$$

Agora, para continuar com a aproximação de Laplace, precisamos achar a matriz Hessiana da posteriori de f dado X e y . A matriz Hessiana é dada por

$$H = \nabla^2 \ln p(f|X, y) = -W - K^{-1} \quad (29)$$

onde W é uma matriz diagonal com entradas $W_{nn} = \sigma_n(1 - \sigma_n)$. A aproximação de Laplace nos diz que a posteriori de f dado X e y pode ser aproximada por uma Gaussiana centrada na moda m com covariância $\Sigma = -H^{-1} = (K^{-1} + W)^{-1}$. Sabendo que $p(f|X, y) \approx \mathcal{N}(m, \Sigma)$, temos que:

$$p(y^*|X, y, x^*) = \int \underbrace{p(y^*|f^*)}_{\mathcal{N}(f^*(x^*), \sigma^2)} \underbrace{p(f^*|X, y, x^*)}_{\mathcal{N}(m', \Sigma')} df^* \quad (30)$$

m' e Σ' representam as mesmas contas que fizemos antes, porém adicionando o ponto x^* no conjunto de dados. A integral acima não tem solução analítica, então podemos utilizar métodos de amostragem para aproximar a integral.

Graph Neural Networks

Toda essa seção é um resumo e escolha dos pontos mais importantes do artigo [1]. Se você quiser se aprofundar mais no assunto, recomendo fortemente a leitura do artigo original. Um ótimo vídeo para ver antes de ler esse resumo é o vídeo do canal Alex Foo [2]

4.1 Introdução

Graph neural networks (GNNs) são uma classe de redes neurais projetadas para trabalhar com dados estruturados em grafos. Diferentemente das redes neurais tradicionais, que operam em dados tabulares ou sequenciais, as GNNs são capazes de capturar a complexidade das relações entre os nós de um grafo, permitindo a modelagem de interações complexas e dependências entre os elementos do grafo.

4.2 Notações

Para facilitar a compreensão, vamos definir algumas notações comuns usadas em GNNs:

- $G = (V, E)$: Um grafo onde V é o conjunto de nós e E é o conjunto de arestas.
- A : Matriz de adjacência do grafo, onde $A_{\{ij\}} = 1$ se houver uma aresta entre os nós i e j , e 0 caso contrário.
- X : Matriz de características dos nós, onde cada linha representa as características de um nó. (Por exemplo, x_i pode ser o conjunto **idade**, **peso**, **altura** de uma pessoa representada pelo nó i).
- Normalmente, é utilizada a matriz de adjacência normalizada com self loops, dada por $A' = (D + I)^{-\frac{1}{2}}(A + I)(D + I)^{-\frac{1}{2}}$, onde D é a matriz diagonal de grau dos nós e I é a matriz identidade onde $D_{ii} = \sum_j A_{ij} = \delta(v_i) = \text{Grau de } v_i$

4.3 Usos de GNNs

Classificação de nós. Seja $\mathcal{Y} = \{1, \dots, L\}$ um conjunto finito de classes, $G = (V, E)$ um grafo e $V_l \subseteq V$ um subconjunto de nós anotados com classes em $\mathcal{Y}$. Classificação de nós consiste no problema de classificar os nós em $V_l^c = V \setminus V_l$. Como exemplo, sistemas de detecção de fraude na Internet objetivam verificar a legitimidade da identidade de usuários; para isso, esses sistemas binariamente classificam como legítimo ou fraudulento os nós de uma rede de pessoas que interagem com alguma interface on-line. Existem duas diferenças essenciais entre classificação de nós em um grafo e os cenários canônicos de classificação em problemas de aprendizado supervisionado. Primeiro, supomos que o grafo, e logo seus nós/amostras, é inteiramente observado e que apenas não conhecemos as classes de algum subconjunto dos nós; em contraste, métodos convencionais de classificação não permitem a classificação de amostras não observadas durante o treinamento do modelo —i.e., classificação de nós costuma ser uma tarefa transdutiva, enquanto métodos convencionais focam em aprendizado indutivo. Segundo, as amostras em um grafo são intrinsecamente correlacionadas e então descumprem a típica suposição de independência distribucional assumida pelos métodos historicamente relevantes de classificação - como os modelos lineares; essa inconsistência explica a efetiva inutilidade destes métodos à classificação de nós em grafos e, no passado, incentivou o desenvolvimento de procedimentos que incorporam a estrutura correlacional das amostras em seus mecanismos de inferência. Circunstancialmente, as GNNs exploram a correlação induzida nas amostras pelo seu grafo subjacente e as informações não estruturais para classificar os nós não anotados em um grafo.

Inferência relacional (predição de aresta). Seja $G = (V, E)$ um grafo e suponha que observamos o grafo parcial $\hat{G} = (V, \hat{E})$ com $\hat{E} \subset E$; o objetivo da inferência relacional é identificar

as arestas (relações) não observadas $\hat{E}$. Por exemplo, a estimativa da probabilidade de que um par de indivíduos se conhece em uma mídia social é crucial para aumentar o engajamento dos usuários com a plataforma e corresponde a uma instanciação do problema de inferência

relacional. Em outra direção, a descrição de como as diferentes proteínas interagem para permitir o desenvolvimento de um organismo é um dos problemas fundacionais de biologia molecular e é equivalente à predição de arestas no grafo de interação entre proteínas (chamado de interatoma). Enfaticamente, a inferência relacional, como a classificação de nós, transcende as fronteiras dos algoritmos tradicionais de aprendizagem de máquina ao exigir o tratamento de amostras correlacionadas para identificar as arestas prováveis em um espaço combinatoriamente grande de arestas possíveis. Em contraste, as GNNs eficientemente utilizam a topologia da rede e os atributos dos nós para precisamente inferir a existência de arestas de G não observadas em $\hat{G}$.

Classificação e regressão de grafos. Alguns problemas exigem o tratamento de bases de dados relacionais em que as instâncias são objetos representados como grafos. O químico que almeja enumerar os efeitos colaterais de determinado medicamento, por exemplo, está tipicamente equipado com um conjunto de outros medicamentos com efeitos colaterais metabolicamente reconhecíveis; e cada medicamento é epistemicamente representado por uma estrutura molecular equivalente a um grafo. Esta categoria de problemas de inferência em grafos é a mais similar e receptiva à abordagem tradicional de aprendizagem de máquina; neste caso, cada grafo corresponde a uma amostra independente e presumivelmente identicamente distribuída as outras. A dificuldade incide na geração de representações vetoriais suficientemente informativas dos grafos para maximizar a eficácia de procedimentos de classificação e de regressão subsequentemente aplicados a estas representações. Notadamente, as redes neurais para grafos naturalmente aprendem representações latentes dos nós que podem ser sucessivamente agregadas e então exploradas em algoritmos de inferência canônicos de aprendizagem de máquina.

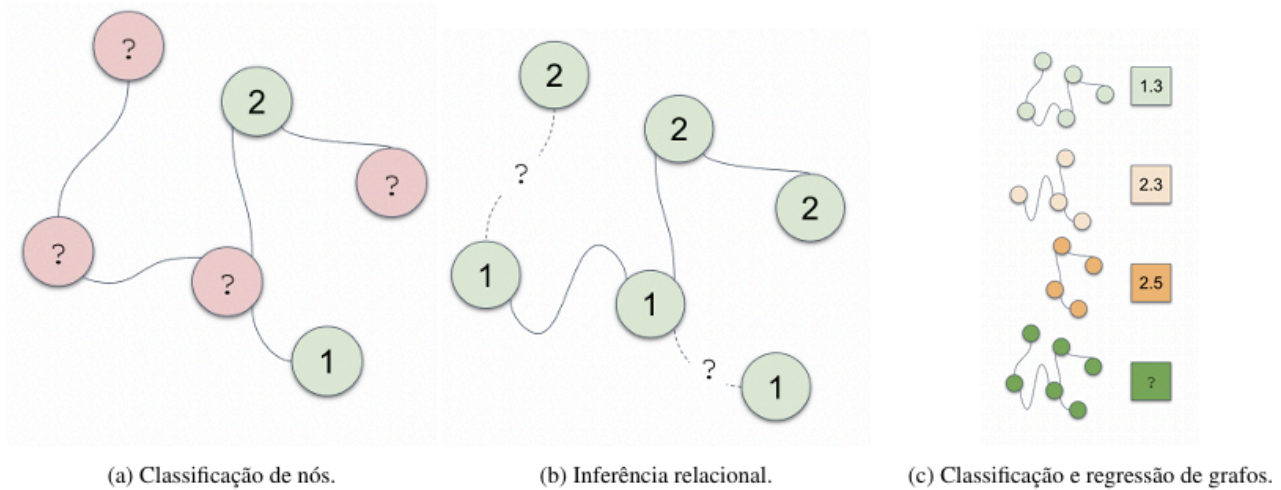


Figura 3: Representação visual de cada um dos três usos de GNNs discutidos

4.4 Passagem de Mensagem

As redes em grafo funcionam de forma que as informações de cada nó são passadas para seus vizinhos, que por sua vez passam as informações para seus vizinhos, e assim por diante. Esse processo é chamado de **passagem de mensagem** (message passing). A passagem de mensagem é um processo iterativo que ocorre em T rodadas, onde T é um hiperparâmetro do modelo. Em cada rodada t , cada nó v recebe mensagens de seus vizinhos $\mathcal{N}(v)$ e atualiza seu estado com base nessas mensagens. Podemos dividir o processo aplicado à cada nó como:

$$\begin{aligned}
 m_v^{(t)} &= \text{AGGREGATE}^{(t)}\left(\left\{h_u^{(t-1)}, \forall u \in \mathcal{N}(v)\right\}\right) & \forall v \in V \\
 h_v^{(t)} &= \text{UPDATE}^{(t)}\left(h_v^{(t-1)}, m_v^{(t)}\right) & \forall v \in V
 \end{aligned} \tag{31}$$

de forma que $h_v^{(0)} = x_v$

As funções AGGREGATE e UPDATE são funções que variam dependendo da implementação, de forma que diferentes implementações de GNNs podem ser obtidas. A função AGGREGATE é responsável por agregar as informações dos vizinhos de um nó, enquanto a função UPDATE é responsável por atualizar o estado do nó com base nas informações agregadas. A escolha dessas funções é crucial para o desempenho da GNN e pode ser feita de várias maneiras, incluindo somas, médias, máximos ou redes neurais.

Podemos reformular de forma mais compacta a passagem de mensagem definindo:

$$\begin{aligned} H^{(t)} &= \begin{pmatrix} - & h_1^{(t)} & - \\ & \vdots & \\ - & h_{|V|}^{(t)} & - \end{pmatrix} \\ M^{(t)} &= \begin{pmatrix} - & m_1^{(t)} & - \\ & \vdots & \\ - & m_{|V|}^{(t)} & - \end{pmatrix} \end{aligned} \quad (32)$$

então reescrevemos os passos anteriores como

$$\begin{aligned} M^{(t)} &= \text{AGGREGATE}^{(t)}(A, H^{(t-1)}) \\ H^{(t)} &= \text{UPDATE}^{(t)}(H^{(t-1)}, M^{(t)}) \end{aligned} \quad (33)$$

4.5 Graph Convolutional Network (GCN)

Popularizou as GNNs por sua simplicidade. É um modelo baseado em message-passing, onde a função de agregação é uma média ponderada dos vizinhos de um nó e a função de atualização é uma rede neural simples. A GCN é definida como:

$$\begin{aligned} m_v^{(t)} &= \sum_{u \in \mathcal{N}(v)} \frac{h_u^{(t-1)}}{\sqrt{\bar{d}_u \bar{d}_v}} \quad \forall v \in V \\ h_v^{(t)} &= \sigma \left(\left(\frac{1}{\bar{d}_v} h_v^{(t-1)} + m_v^{(t)} \right) \Theta_t \right) \quad \forall v \in V \end{aligned} \quad (34)$$

onde Θ_t é uma matriz de pesos aprendida durante o treinamento e $\bar{d}_v$ é o grau do nó v com self-loops. A função de ativação σ é geralmente uma função não-linear como ReLU ou sigmoid. Podemos reescrever a GCN de forma matricial como:

$$H^{(t)} = \sigma \left(D^{-\frac{1}{2}} A D^{-\frac{1}{2}} H^{(t-1)} \Theta_t \right) \quad (35)$$

Convolutional Neural Networks (CNN)

5.1 Introdução

Dentro do campo de aprendizado de máquina, redes neurais convolucionais (CNNs) são uma classe de redes neurais artificiais projetadas para processar dados com uma estrutura de grade, como imagens. Elas são inspiradas na organização do córtex visual dos animais e são particularmente eficazes em tarefas de visão computacional, como reconhecimento de objetos, detecção de objetos e segmentação de imagens.

O campo de visão computacional tem sido um dos principais impulsionadores do desenvolvimento de CNNs, com aplicações em reconhecimento facial, análise de imagens médicas, veículos autônomos e muito mais. As CNNs são capazes de aprender automaticamente características hierárquicas dos dados, permitindo que elas capturem padrões complexos e invariantes a transformações, como rotação e escala. Algumas aplicações de machine learning no campo da visão computacional são:

- **Classificação de imagens:** Identificar a categoria a que uma imagem pertence, como classificar fotos de animais em diferentes espécies.
- **Detecção de objetos:** Localizar e identificar objetos específicos dentro de uma imagem, como detectar carros, pedestres ou sinais de trânsito em imagens de ruas.
- **Segmentação de imagens:** Dividir uma imagem em regiões significativas, como segmentar diferentes órgãos em imagens médicas para análise diagnóstica.
- **Reconhecimento facial:** Identificar ou verificar a identidade de uma pessoa com base em sua imagem facial, utilizado em sistemas de segurança e autenticação.
- **Síntese:** Gerar novas imagens a partir de descrições textuais ou de outras imagens, como criar imagens realistas de pessoas ou objetos que não existem.

5.2 Imagens como dados

Podemos interpretar imagens como dados estruturados em uma grade bidimensional, onde cada pixel representa uma unidade de informação. Cada pixel possui valores que representam a intensidade da cor em diferentes canais (como vermelho, verde e azul para imagens RGB). Essa estrutura de grade permite que as CNNs explorem a relação espacial entre os pixels, capturando padrões locais e hierárquicos.

$$I \in \mathbb{R}^{H \times W \times C} \quad (36)$$

onde H , W e C representam a altura, largura e número de canais da imagem, respectivamente. No caso, se a imagem é colorida, $C = 3$ e se ela é preto e branco, $C = 1$ (O que podemos entender como uma matriz com dimensões $H \times W$)

5.3 Filtros

As CNNs são interessantes. Se tentássemos treinar uma rede neural comum usando uma imagem, seria inviável, pois a rede seria MUITO grande. Imagine uma imagem de, por exemplo, 1000×1000 pixels. Se tentássemos treinar uma rede neural comum usando essa imagem, teríamos 1.000.000 de entradas (uma para cada pixel), além de que se for colorida, seria 3.000.000. Isso resultaria em uma rede neural com milhões de parâmetros, tornando o treinamento extremamente difícil e propenso a overfitting. Para tentar contornar esse problema, as CNNs utilizam de diversas abordagens para, dentro da arquitetura, capturar propriedades específicas de imagens:

- **Hierarquia:** Elementos em imagens possuem uma hierarquia natural. Por exemplo, uma imagem de um rosto humano pode ser decomposta em partes como olhos, nariz e boca, que por sua vez podem ser decompostas em características mais simples, como bordas e texturas. As CNNs são projetadas para capturar essas hierarquias de características, permitindo que a rede aprenda representações cada vez mais complexas à medida que avança pelas camadas.

- **Localidade:** As CNNs exploram a localidade das imagens, ou seja, a ideia de que pixels próximos uns dos outros estão mais relacionados do que pixels distantes. Isso é feito através do uso de filtros convolucionais, que operam em pequenas regiões da imagem, permitindo que a rede capture padrões locais e invariantes a transformações.
- **Equivariância:** As CNNs são projetadas para serem equivariantes a translações, o que significa que se um objeto na imagem for deslocado, a rede ainda será capaz de reconhecê-lo. Isso é alcançado através do uso de operações de convolução e pooling, que permitem que a rede aprenda características independentes da posição do objeto na imagem.
- **Invariância:** As CNNs também podem ser projetadas para serem invariantes a certas transformações, como rotação e escala. Isso é feito através do uso de técnicas como data augmentation, que aumentam a diversidade do conjunto de treinamento, e camadas de pooling, que reduzem a sensibilidade da rede a pequenas variações na posição e tamanho dos objetos.

5.3.1 Capturando Localidade

Por simplicidade, no momento vamos assumir que nossas imagens estão na escala de cinza (são matrizes no $\mathbb{R}^{H \times W}$). Queremos, de alguma forma, capturar a localidade das imagens. Intuitivamente, podemos pensar em, de alguma forma, resumir uma região da imagem em um único valor. Por exemplo, podemos pegar uma região de 3×3 pixels e calcular a média dos valores dos pixels dessa região. Isso nos daria um único valor representando a intensidade média da região. No entanto, essa abordagem simples não captura padrões mais complexos, como bordas ou texturas

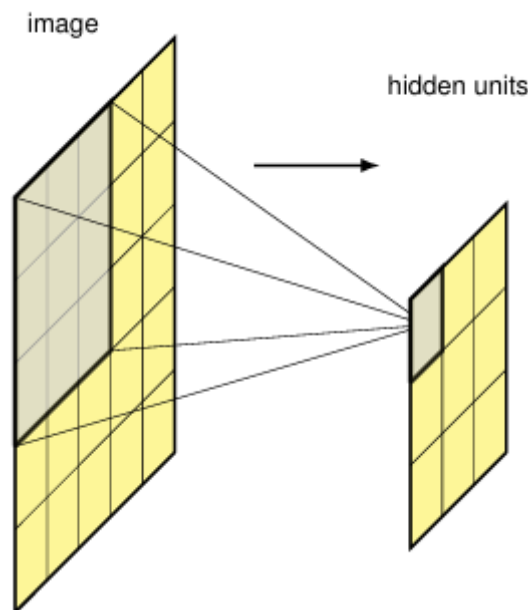


Figura 4: Representação visual de como podemos capturar a localidade de uma imagem usando uma região de 3×3 pixels

Uma forma mais interessante que reflete o que fizemos até o momento, é aplicar uma matriz de pesos à esses pixels.

$$z = \text{ReLU}(w^T x + w_0) \quad (37)$$

onde x é o vetor de pixels da região, w é o vetor de pesos e w_0 é o viés. Essa abordagem permite que a rede aprenda padrões mais complexos, como bordas ou texturas, ao invés de apenas calcular a média da região. Além disso, podemos aplicar diferentes matrizes de pesos a diferentes regiões da imagem, permitindo que a rede aprenda diferentes padrões em diferentes partes da imagem. Esses filtros também são chamados popularmente de **kernels** e são aplicados a toda a imagem,

permitindo que a rede aprenda padrões invariantes à posição do objeto na imagem. A operação de aplicar um filtro a uma região da imagem é chamada de **convolução**, e é a base das CNNs.

5.4 Equivariância em Translação

Imagine que estamos tentando identificar um rosto em uma imagem, se o rosto estiver em uma posição diferente na imagem, a rede ainda deve ser capaz de reconhecê-lo. Nossa rede neural precisa ser capaz de capturar essa propriedade, mas como? Se um filtro é capaz de identificar uma borda em uma região da imagem, ele deve ser capaz de identificar a mesma borda em qualquer outra região da imagem. Isso significa que os filtros devem ser aplicados a toda a imagem, permitindo que a rede aprenda padrões invariantes à posição do objeto na imagem. Essa propriedade é chamada de **equivariância em translação**, e é uma das principais vantagens das CNNs em relação às redes neurais tradicionais.

Definição 5.4.1 (Feature Map/Convolução): Para uma imagem I com intensidades de pixel $I(j, k)$ e um filtro K com valores $K(l, m)$, a feature map C tem valores de ativação:

$$C(j, k) = \sum_l \sum_m I(j + l, k + m) K(l, m) \quad (38)$$

é comum representar essa operação como $C = I * K$, onde $*$ denota a operação de convolução. A feature map é uma representação da imagem original, onde cada valor de ativação representa a presença de um padrão específico na região correspondente da imagem. Vale também ressaltar que essa operação, mesmo se chamando convolução, difere da convolução matemática tradicional.

Se $I \in \mathbb{R}^{H \times W}$ e $K \in \mathbb{R}^{h \times w}$, então $C \in \mathbb{R}^{(H-h+1) \times (W-w+1)}$

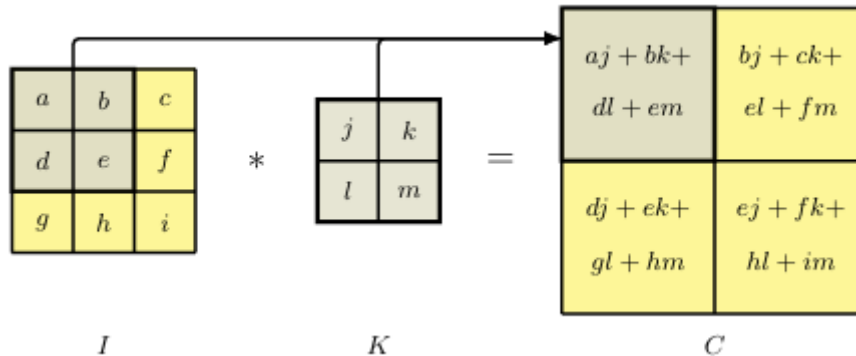


Figura 5: Representação visual da operação de convolução, onde a feature map C é obtida aplicando o filtro K à imagem I

5.5 Padding

Podemos ver da Figura 5 que a feature map C é menor que a imagem original I . Isso ocorre porque a convolução é aplicada apenas às regiões da imagem onde o filtro pode ser completamente sobreposto. Para evitar essa redução de tamanho, podemos aplicar **padding** à imagem original, adicionando uma borda de zeros ao redor da imagem após uma normalização (Assim, o 0 representa o valor médio de pixel da imagem). Isso permite que o filtro seja aplicado a todas as regiões da imagem, incluindo as bordas, resultando em uma feature map do mesmo tamanho que a imagem original. Se minha imagem I tem dimensões $H \times W$ e o filtro K tem dimensões $M \times M$, então a feature map C terá dimensões $(H - M + 1) \times (W - M + 1)$, se eu aplicar um padding de tamanho P , então a feature map C terá dimensões $(H - M + 1 + 2P) \times (W - M + 1 + 2P)$.

Isso se chama uma **padding válido**. Quando o padding é escolhido de forma que o tamanho da feature map seja o mesmo que o tamanho da imagem original, chamamos de **padding completo** ($P = (M - 1)/2$). O padding é uma técnica importante em CNNs, pois permite que a rede aprenda padrões em todas as regiões da imagem, incluindo as bordas.

0	0	0	0	0	0
0	X_{11}	X_{12}	X_{13}	X_{14}	0
0	X_{21}	X_{22}	X_{23}	X_{24}	0
0	X_{31}	X_{32}	X_{33}	X_{34}	0
0	X_{41}	X_{42}	X_{43}	X_{44}	0
0	0	0	0	0	0

Figura 6: Padding de 1 pixel aplicado à uma imagem 4×4 , transformando ela em uma imagem 6×6 com uma borda de zeros ao redor da imagem original

5.6 Convoluções com Stride

Além do padding, outra técnica importante em CNNs é o **stride**, que é o passo. O stride define quantos pixels o filtro se move a cada aplicação da convolução. Por exemplo, se o stride for 1, o filtro se move um pixel de cada vez, enquanto se o stride for 2, o filtro se move dois pixels de cada vez. O uso do stride permite que a rede aprenda padrões em diferentes escalas e resoluções, além de reduzir o tamanho da feature map resultante. Se minha imagem I tem dimensões $H \times W$ e o filtro K tem dimensões $M \times M$, e eu aplico o mesmo passo S tanto verticalmente quanto horizontalmente, e eu apliquei um **padding completo**, então a dimensão do feature map será:

$$\left\lfloor \frac{H + 2P - M}{S} - 1 \right\rfloor \times \left\lfloor \frac{W + 2P - M}{S} - 1 \right\rfloor \quad (39)$$

5.7 Convolução Multidimensional

Até o momento, falamos das operações em matrizes bidimensionais (2 valores), porém, a maioria das imagens são coloridas, então teríamos 3 matrizes, uma para cada canal de cor. Expandir essa operação para múltiplos canais é relativamente simples. Se minha imagem I tem dimensões $H \times W \times C$ (altura, largura e quantidade de canais) e o filtro K tem dimensões $M \times M \times C$, eu vou utilizar cada feature map $M \times M$ com seu respectivo canal da imagem, e somar os resultados. A feature map resultante terá dimensões $(H - M + 1) \times (W - M + 1)$, e cada valor de ativação representará a presença de um padrão específico na região correspondente da imagem, considerando todos os canais de cor. Essa operação é chamada de **convolução multidimensional** e é fundamental para o processamento de imagens coloridas em CNNs.

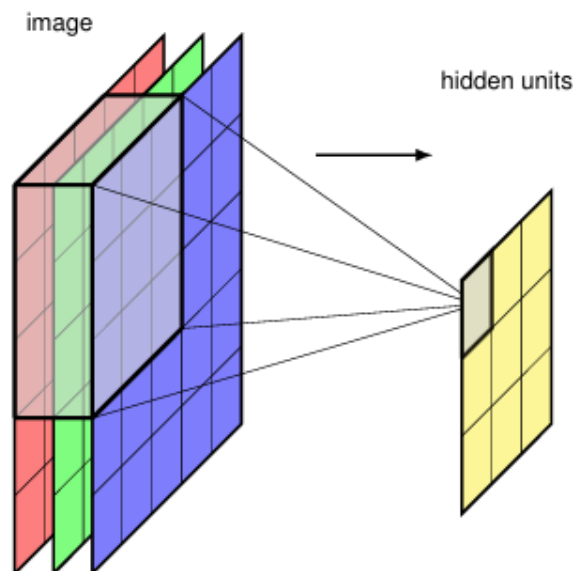


Figura 7: Representação visual da operação de convolução multidimensional

No entanto, essa feature map que formamos, é capaz de detectar apenas um certo padrão de formatos. Por exemplo, ela só pode detectar olhos, mas não pode detectar narizes. Para resolver esse problema, podemos utilizar múltiplos filtros, cada um capaz de detectar um padrão diferente. Dessa forma, dado uma imagem de tamanho $H \times W \times C$, o filtro agora terá tamanho $M \times M \times C \times C_{OUT}$ onde C_{OUT} é a quantidade de feature maps resultantes. Cada feature map possui seu próprio bias, dessa forma, o número total de parâmetros do modelo será $C_{OUT}(M^2C + 1)$.

5.8 Pooling

Nós vimos anteriormente como obter equivariância à translação, porém, em certas aplicações, queremos que a mesma imagem, mesmo que transladada, seja classificada da mesma forma. Para isso, podemos utilizar uma operação chamada **pooling**, que é uma operação de downsampling que reduz a dimensionalidade da feature map, mantendo as informações mais importantes. Existem diferentes tipos de pooling, como **max pooling**, **average pooling** e **global pooling**.

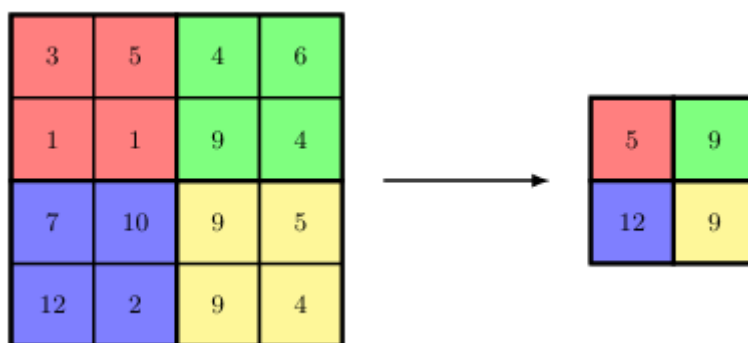


Figura 8: Operação de max-pooling em um feature map 4×4 com uma janela de 2×2 e stride de 2, resultando em um feature map 2×2 . A operação de max pega o valor máximo da janela de pooling

Seguindo o exemplo da Figura 8, podemos ver que a operação de pooling reduz a dimensionalidade da feature map, mantendo as informações mais importantes. A operação de pooling é importante em CNNs, pois permite que a rede aprenda padrões invariantes à posição do objeto na imagem, além de reduzir o número de parâmetros do modelo e evitar overfitting. Além de max-pooling, também podemos utilizar **average pooling**, que calcula a média dos valores da janela de pooling, e **global pooling**, que calcula a média ou o máximo de toda a feature map. A escolha do tipo de pooling depende da aplicação e do problema em questão.

5.9 Arquiteturas

Mas como podemos combinar todas essas operações para formar uma rede neural convolucional? A resposta é através de arquiteturas. Uma arquitetura de CNN é composta por várias camadas, cada uma com suas próprias operações de convolução, pooling e funções de ativação. As camadas são organizadas em uma sequência, onde a saída de uma camada é a entrada da próxima camada. As arquiteturas podem variar em profundidade, largura e complexidade, dependendo do problema e da aplicação.

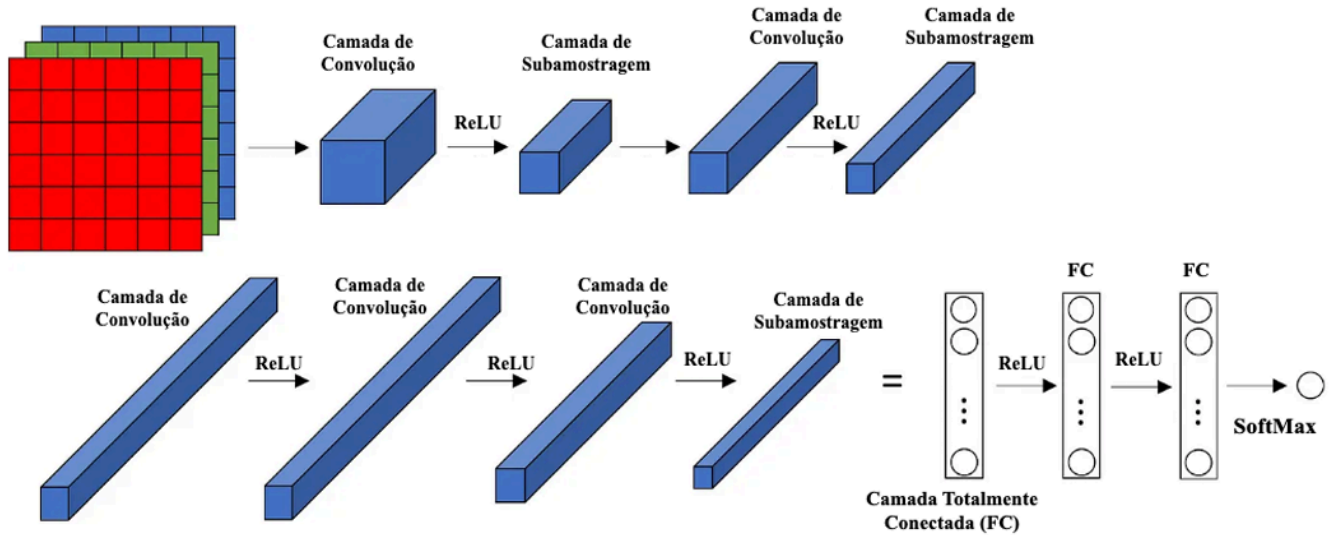


Figura 9: Representação visual da arquitetura da AlexNet, uma das primeiras CNNs a alcançar sucesso em tarefas de visão computacional. A AlexNet é composta por várias camadas de convolução, pooling e funções de ativação, além de camadas totalmente conectadas no final

5.10 Os Gradientes

Não basta mostrar arquiteturas e métodos usados em CNN se não sabemos como treiná-las. Para isso, precisamos entender como calcular os gradientes das operações de convolução e pooling. O cálculo dos gradientes é feito através do algoritmo de backpropagation (também muito utilizado o automatic differentiation).

5.10.1 Convolução sem Padding e Stride

Vamos primeiro olhar a derivação dos gradientes no caso mais fácil, que é com uma imagem com escalas de cinza, onde toda a imagem é representada por uma única matriz 2D.

Definição 5.10.1.1 (Convolução/Correlação Cruzada 2D sem padding e stride): Seja a imagem $X \in \mathbb{R}^{H \times W}$ e o filtro $K \in \mathbb{R}^{M \times N}$, definimos a saída da camada convolucional, sem padding e sem stride como:

$$(X * K)_{ij} = Y_{ij} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X_{i+m, j+n} K_{mn} + b \quad (40)$$

para

$$0 \leq i \leq H - M \quad 0 \leq j \leq W - N \quad (41)$$

considere que estamos trabalhando com o gradiente em cima de uma função de perda L , por exemplo, se estamos usando a CNN para fazer a classificação de imagens, podemos usar a função de perda **cross-entropy**. Defina também:

$$\delta_{ij} = \frac{\partial L}{\partial Y_{ij}} \quad (42)$$

Teorema 5.10.1.1 (Gradiente da Convolução Discreta 2D sem padding e stride): O gradiente da perda em relação ao coeficiente K_{mn} é dado por:

$$\frac{\partial L}{\partial K_{mn}} = \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \delta_{ij} X_{i+m,j+n} \quad (43)$$

para

$$0 \leq m \leq M-1 \quad 0 \leq n \leq N-1 \quad (44)$$

ou, equivalentemente, podemos escrever de forma matricial como:

$$\nabla_K L = X * \delta \quad (45)$$

onde $\delta \in \mathbb{R}^{H-M+1 \times W-N+1}$ e

$$\delta = \begin{pmatrix} - & \delta_{0,0} & - & \delta_{0,1} & - & \dots & - & \delta_{0,W-N} & - \\ & \vdots & & & & & & & \\ - & \delta_{H-M,0} & - & \delta_{H-M,1} & - & \dots & - & \delta_{H-M,W-N} & - \end{pmatrix} \quad (46)$$

Demonstração: Pela regra da cadeia, temos que

$$\frac{\partial L}{\partial K_{mn}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \frac{\partial Y_{ij}}{\partial K_{mn}} \quad (47)$$

e, por definição

$$Y_{ij} = \sum_{u,v} X_{i+u,j+v} K_{uv} + b \quad (48)$$

logo:

$$\frac{\partial Y_{ij}}{\partial K_{mn}} = X_{i+m,j+n} \quad (49)$$

substituindo na equação anterior, temos que

$$\frac{\partial L}{\partial K_{mn}} = \sum_{i,j} \delta_{ij} X_{i+m,j+n} \quad (50)$$

□

Teorema 5.10.1.2 (Gradiente do Bias): Temos que:

$$\frac{\partial L}{\partial b} = \sum_{ij} \delta_{ij} \quad (51)$$

Demonstração: Novamente, pela regra da cadeia, temos que

$$\frac{\partial L}{\partial b} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \frac{\partial Y_{ij}}{\partial b} \quad (52)$$

e como vimos anteriormente, pela definição, temos que

$$\frac{\partial Y_{ij}}{\partial b} = 1 \quad (53)$$

substituindo na equação anterior, temos que

$$\frac{\partial L}{\partial b} = \sum_{i,j} \delta_{ij} \quad (54)$$

□

Teorema 5.10.1.3 (Gradiente da Imagem): O gradiente da perda em relação à imagem X é dado por:

$$\frac{\partial L}{\partial X_{ij}} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \delta_{i-m, j-n} K_{mn} \quad (55)$$

para

$$0 \leq i \leq H-1 \quad 0 \leq j \leq W-1 \quad (56)$$

ou, equivalentemente, podemos escrever de forma matricial como:

$$\nabla_X L = \delta * K^{\text{flip}} \quad (57)$$

onde $\delta \in \mathbb{R}^{H-M+1 \times W-N+1}$ e K^{flip} é a imagem da kernel K rotacionada por 180 graus, ou seja:

$$K_{mn}^{\text{flip}} = K_{M-1-m, N-1-n} \quad (58)$$

Demonstração: Vamos fixar uma coordenada (a, b) e calcular o gradiente da perda em relação à imagem X nessa coordenada. Pela regra da cadeia, temos que:

$$\frac{\partial L}{\partial X_{ab}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \frac{\partial Y_{ij}}{\partial X_{ab}} \quad (59)$$

como vimos anteriormente, pela definição, temos que

$$Y_{ij} = \sum_{u,v} X_{i+u, j+v} K_{uv} + \text{bias} \quad (60)$$

temos então que

$$\frac{\partial Y_{ij}}{\partial X_{ab}} = K_{a-i, b-j} \quad (61)$$

sempre que os índices estão no suporte do kernel. Logo,

$$\frac{\partial L}{\partial X_{ab}} = \sum_{i,j} \delta_{ij} K_{a-i, b-j} \quad (62)$$

e por que essa expressão equivale a K^{flip} ? Pois, se definirmos $K_{ij}^{\text{flip}} = K_{M-1-i, N-1-j}$, então podemos reescrever a expressão acima como:

$$\frac{\partial L}{\partial X_{ab}} = \sum_{i,j} \delta_{a-i, b-j} K_{ij}^{\text{flip}} \quad (63)$$

□

5.10.2 Convolução com Padding e Stride

Definição 5.10.2.1 (Padding): Padding pode ser definido como uma função $T : \mathbb{R}^{H \times W} \rightarrow \mathbb{R}^{H+2P \times W+2P}$, onde P é o tamanho do padding que adiciona P linhas e P colunas de zeros ao redor da imagem original

Teorema 5.10.2.1 (Padding é Linear): Seja $X, Y \in \mathbb{R}^{H \times W}$ e $T(X) \in \mathbb{R}^{H+2P \times W+2P}$ o padding de X , então temos que:

$$T(\alpha X + \beta Y) = \alpha T(X) + \beta T(Y) \quad (64)$$

Demonstração: Primeiro, vamos mostrar que T é linear em vetores, e então, mostrar que a operação também é linear em matrizes. Seja $x \in \mathbb{R}^m$ e $\hat{x} = T(x) \in \mathbb{R}^{m+2P}$ o padding de x , temos que a operação faz:

$$\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{pmatrix} \mapsto \begin{pmatrix} 0 \\ \vdots \\ 0 \\ x_1 \\ \vdots \\ x_m \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad (65)$$

No entanto, perceba que, se definirmos a matriz:

$$M = \begin{pmatrix} \mathbf{0} \\ I \\ \mathbf{0} \end{pmatrix} \in \mathbb{R}^{(m+2P) \times m} \quad (66)$$

onde $\mathbf{0} \in \mathbb{R}^{P \times m}$ e a identidade $I \in \mathbb{R}^{m \times m}$, então podemos definir:

$$\hat{x} = T(x) = Mx \quad (67)$$

então podemos definir o padding de matrizes como:

$$T(X) = \begin{pmatrix} \mathbf{0} & \left| \begin{array}{c} T(x_1) \\ \vdots \\ T(x_W) \end{array} \right| & \mathbf{0} \end{pmatrix} \quad (68)$$

esses novos 0 são bloco de matrizes de zeros de tamanho $H \times P$. Logo, temos que:

$$T(X) = (\mathbf{0} \ M \ \mathbf{0})X \quad (69)$$

onde M é matriz $(m + 2P) \times m$ que vimos e $\mathbf{0}$ é uma matriz de zeros de tamanho $H \times P$. Logo, temos no final uma matriz de tamanho $(H + 2P) \times (W + 2P)$ e, como T é uma multiplicação de matrizes, temos que T é linear. $\square$

Definição 5.10.2.2 (Stride): Seja $S \in \mathbb{N}$ o stride. Definimos o operador de stride

$$D_S : \mathbb{R}^{H \times W} \rightarrow \mathbb{R}^{H' \times W'} \quad (70)$$

onde

$$H' = \left\lfloor \frac{H-1}{S} \right\rfloor + 1 \quad (71)$$

e

$$W' = \left\lfloor \frac{W-1}{S} \right\rfloor + 1 \quad (72)$$

tal que

$$(D_S(X))_{i,j} = X_{iS,jS}. \quad (73)$$

Em outras palavras, o operador mantém apenas as entradas espaçadas de S posições.

Teorema 5.10.2.2 (Stride é Linear): Seja

$$D_S : \mathbb{R}^{H \times W} \rightarrow \mathbb{R}^{H' \times W'} \quad (74)$$

o operador de stride de tamanho S .

Então

$$D_S(\alpha X + \beta Y) = \alpha D_S(X) + \beta D_S(Y) \quad (75)$$

para quaisquer

$$X, Y \in \mathbb{R}^{H \times W} \quad (76)$$

e

$$\alpha, \beta \in \mathbb{R}. \quad (77)$$

Demonstração: Primeiro consideremos o caso vetorial.

Seja

$$x = (x_0, x_1, \dots, x_{n-1})^T \in \mathbb{R}^n. \quad (78)$$

O operador de stride mantém apenas as coordenadas

$$0, S, 2S, \dots \quad (79)$$

Assim,

$$D_S(x) = (x_0, x_S, x_{2S}, \dots)^T. \quad (80)$$

Definamos a matriz

$$M_S = \begin{pmatrix} 1 & 0 & 0 & 0 & \dots \\ 0 & \dots & 1 & 0 & \dots \\ \dots & \dots & \dots & \dots & \dots \end{pmatrix} \quad (81)$$

cujas linhas possuem exatamente um elemento igual a 1 nas posições

$$0, S, 2S, \dots \quad (82)$$

e zero nas demais.

Então

$$D_S(x) = M_S x. \quad (83)$$

Logo,

$$D_S(\alpha x + \beta y) = M_S(\alpha x + \beta y) \quad (84)$$

$$= \alpha M_S x + \beta M_S y \quad (85)$$

$$= \alpha D_S(x) + \beta D_S(y). \quad (86)$$

Portanto D_S é linear em vetores. $\square$

Definição 5.10.2.3 (Convolução/Correlação Cruzada 2D com padding e stride): Seja $X \in \mathbb{R}^{H \times W}$, $K \in \mathbb{R}^{M \times N}$, P o padding, S o stride e $\tilde{X} = T(X)$ a imagem com padding, então a saída da camada convolucional é dada por:

$$Y_{ij} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \tilde{X}_{i \cdot S + m, j \cdot S + n} K_{mn} + b \quad (87)$$

Teorema 5.10.2.3 (Gradiente da Convolução Discreta 2D com padding e stride): O gradiente da perda em relação ao coeficiente K_{mn} é dado por:

$$\frac{\partial L}{\partial K_{mn}} = \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \delta_{ij} \tilde{X}_{i \cdot S + m, j \cdot S + n} \quad (88)$$

para

$$0 \leq m \leq M-1 \quad 0 \leq n \leq N-1 \quad (89)$$

ou, equivalentemente, podemos escrever de forma matricial como:

$$\nabla_K L = \tilde{X} * \delta \quad (90)$$

onde $*$ denota a convolução cruzada com stride S

Demonstração: Pela regra da cadeia, temos que

$$\frac{\partial L}{\partial K_{mn}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \frac{\partial Y_{ij}}{\partial K_{mn}} \quad (91)$$

e, por definição

$$Y_{ij} = \sum_{u,v} \tilde{X}_{i \cdot S+u, j \cdot S+v} K_{uv} + b \quad (92)$$

logo:

$$\frac{\partial Y_{ij}}{\partial K_{mn}} = \tilde{X}_{i \cdot S+m, j \cdot S+n} \quad (93)$$

substituindo na equação anterior, temos que

$$\frac{\partial L}{\partial K_{mn}} = \sum_{i,j} \delta_{ij} \tilde{X}_{i \cdot S+m, j \cdot S+n} \quad (94)$$

□

Teorema 5.10.2.4 (Gradiente da Entrada com Stripe): Defina o operador de expansão

$$U_S(\delta) : \mathbb{R}^{H \times W} \rightarrow \mathbb{R}^{(H-1) \cdot (S-1)+1 \times (W-1) \cdot (S-1)+1} \quad (95)$$

obtido inserindo $S - 1$ linhas e colunas de zeros entre cada linha e coluna de δ . Então, o gradiente da perda em relação à imagem X é dado por:

$$\nabla_{\tilde{X}} L = U_S(\delta) * K^{\text{flip}} \quad (96)$$

Exemplo: Suponha $S = 2$ e

$$\delta = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad (97)$$

então

$$U_2(\delta) = \begin{pmatrix} a & 0 & b \\ 0 & 0 & 0 \\ c & 0 & d \end{pmatrix} \quad (98)$$

logo

$$\nabla_{\tilde{X}} L = U_2(\delta) * K^{\text{flip}} \quad (99)$$

A intuição é que, quando o stride é 2, a convolução só visita

$$(0, 0), (0, 2), (2, 0), (2, 2) \quad (100)$$

As posições intermediárias nunca participam do forward. Por isso surgem os zeros na expansão, pois essas posições intermediárias não contribuem para o gradiente da entrada.

5.11 Otimizações Computacionais

Fizemos algumas definições e teoremas sobre convolução, mas não falamos sobre como implementá-las de forma eficiente. A implementação direta das operações de convolução e pooling

pode ser ineficiente, especialmente para imagens grandes e redes profundas. Para melhorar a eficiência computacional, podemos utilizar certas técnicas

5.11.1 im2col

Temos um problema, na convolução comum, temos:

$$Y_{ij} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \tilde{X}_{i \cdot S+m, j \cdot S+n} K_{mn} + b \quad (101)$$

essa operação já tem complexidade $O(MN)$, e isso é apenas para uma das entradas, tendo em mente que faremos isso para outras $H \times W$ entradas, a complexidade sobe para $O(HWMN)$, o que é computacionalmente caro. Entretanto, existe uma observação genial que podemos fazer aqui

Considere a imagem:

$$X = \begin{pmatrix} x_{0,0} & x_{0,1} & x_{0,2} & x_{0,3} \\ x_{1,0} & x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,0} & x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,0} & x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \quad (102)$$

e um kernel 3×3 . Em um stride padrão de 1, as janelas visitadas são. Primeira:

$$\begin{pmatrix} x_{0,0} & x_{0,1} & x_{0,2} \\ x_{1,0} & x_{1,1} & x_{1,2} \\ x_{2,0} & x_{2,1} & x_{2,2} \end{pmatrix} \quad (103)$$

Segunda:

$$\begin{pmatrix} x_{0,1} & x_{0,2} & x_{0,3} \\ x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \end{pmatrix} \quad (104)$$

e assim vai. A ideia é transformar cada uma dessas janelas em uma coluna de uma matriz. Primeira janela:

$$\begin{pmatrix} x_{0,0} \\ x_{0,1} \\ x_{0,2} \\ x_{1,0} \\ x_{1,1} \\ x_{1,2} \\ x_{2,0} \\ x_{2,1} \\ x_{2,2} \end{pmatrix} \quad (105)$$

Segunda janela:

$$\begin{pmatrix} x_{0,1} \\ x_{0,2} \\ x_{0,3} \\ x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \end{pmatrix} \quad (106)$$

e assim vai. Então como resultado, vamos ter:

$$X_{\text{col}} = \begin{pmatrix} x_{0,0} & x_{0,1} \\ x_{0,1} & x_{0,2} \\ x_{0,2} & x_{0,3} \\ x_{1,0} & x_{1,1} \\ x_{1,1} & x_{1,2} & \dots \\ x_{1,2} & x_{1,3} \\ x_{2,0} & x_{2,1} \\ x_{2,1} & x_{2,2} \\ x_{2,2} & x_{2,3} \end{pmatrix} \quad (107)$$

e o resultado obtido da convolução será Y_{col} e estará no mesmo estilo de X_{col} . Para isso, precisamos achatar o kernel, de forma que, se nosso kernel tem tamanho 3×3

$$K = \begin{pmatrix} k_{0,0} & k_{0,1} & k_{0,2} \\ k_{1,0} & k_{1,1} & k_{1,2} \\ k_{2,0} & k_{2,1} & k_{2,2} \end{pmatrix} \quad (108)$$

então K_{col} será:

$$K_{\text{col}} = \begin{pmatrix} k_{0,0} \\ k_{0,1} \\ k_{0,2} \\ k_{1,0} \\ k_{1,1} \\ k_{1,2} \\ k_{2,0} \\ k_{2,1} \\ k_{2,2} \end{pmatrix} \quad (109)$$

Logo, teremos que

$$Y_{\text{col}} = K_{\text{col}}^T X_{\text{col}} + b \quad (110)$$

matematicamente, essa operação também não altera nada, pois no backward do filtro

$$\nabla_K L = X_{\text{col}} \delta_{\text{col}}^T \quad (111)$$

e o backward do input

$$\nabla_{X_{\text{col}}} L = K_{\text{col}} \delta_{\text{col}} \quad (112)$$

5.11.2 col2im

Essa operação é mais utilizada para, a partir do gradiente de X_{col} , obtermos o gradiente com respeito de X de volta para continuar as operações do backpropagation

5.11.3 Aplicação como uma Multiplicação de Matrizes

Como vimos, a maior parte das operações de convolução podem ser representadas como multiplicações de matrizes, o que permite que possamos utilizar bibliotecas otimizadas para multiplicação de matrizes, como BLAS e cuBLAS, para acelerar o treinamento das CNNs. Além disso, podemos utilizar técnicas de paralelização e distribuição para treinar redes profundas em grandes conjuntos de dados.

Seja X a imagem de entrada, podemos definir a saída da convolução como:

$$Y = D_S \cdot K \cdot M \cdot D_P \cdot X \quad (113)$$

onde

- D_P é o operador de padding
- M é o operador de im2col
- K é o operador de multiplicação do kernel
- D_S é o operador de stride

Então o backward será simplesmente a operação:

$$\nabla_X L = D_P^* \cdot M^* \cdot K^* \cdot D_S^* \cdot \delta \quad (114)$$

- Crop P^*
- col2im M^*
- Kernel invertido K^*
- Operador de expansão D_S^* (O mesmo definido em Teorema 5.10.2.4)

5.12 Pooling

Para as definições a baixo, vamos considerar janelas já transformadas em colunas após a operação de im2col, e o stride já aplicado. Ou seja, a entrada da operação de pooling será um vetor $x \in \mathbb{R}^m$, de forma que a operação é aplicada em cada coluna da matriz X_{col}

Definição 5.12.1 (Max Pooling): Seja $x \in \mathbb{R}^m$ a saída da camada de max pooling é dada por:

$$Y_{ij} = \max x \quad (115)$$

Teorema 5.12.1 (Não-linearidade): A operação de max pooling é não-linear, ou seja, não podemos expressá-la como uma combinação linear das entradas. Isso significa que a operação de pooling não pode ser representada como uma multiplicação de matrizes, o que dificulta a análise teórica da operação

Definição 5.12.2 (Adjunto do Max Pooling): O adjunto do max pooling consiste em você armazenar a posição da última entrada máxima de cada janela de pooling, e no backward, você propaga o gradiente apenas para essa posição, enquanto as demais posições recebem gradiente zero. Isso garante que o gradiente seja propagado corretamente através da operação de max pooling, permitindo que a rede aprenda padrões invariantes à posição do objeto na imagem

Exemplo:

$$(a \ b \ c \ d) \quad (116)$$

Supondo que b é o maior valor, o max pooling vai retornar

$$b \quad (117)$$

Então no backward, a matriz gerada será:

$$(0 \ \delta \ 0 \ 0) \quad (118)$$

Definição 5.12.3 (Average Pooling): Seja $x \in \mathbb{R}^{k^2}$ a coluna representando uma janela $k \times K$ a saída da camada de average pooling é dada por:

$$y = \left(\frac{1}{k^2} \right) \sum_i^{k^2} x_i \quad (119)$$

Teorema 5.12.2 (Linearidade do Average Pooling): A operação de average pooling é linear, ou seja, podemos expressá-la como uma combinação linear das entradas. Isso significa que a operação de pooling pode ser representada como uma multiplicação de matrizes

Demonstração: Definindo a matriz

$$Q = \frac{1}{k^2} (1 \ 1 \ 1 \ \dots \ 1) \quad (120)$$

podemos escrever

$$y = Qx \quad (121)$$

assim, ainda obtemos seu adjunto como Q^T , espalhando o erro igualmente para todas as camadas

$$Q^T \delta = \frac{1}{k^2} \begin{pmatrix} \delta \\ \delta \\ \delta \\ \dots \\ \delta \end{pmatrix} = \frac{\delta}{k^2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ \dots \\ 1 \end{pmatrix} \quad (122)$$

□

Referências

- [1] T. da Silva, A. H. Souza, and D. Mesquita, “UMA INTRODUÇÃO AMIGÁVEL ÀS REDES NEURAIS PARA GRAFOS,” *Journal of the Brazilian Society on Computational Intelligence*, pp. 1–13, 2021.
- [2] A. Foo, “Graph Neural Networks - a perspective from the ground up.” [Online]. Available: <https://youtu.be/GXhBEj1ZtE8?si=iWDtHtxtLmyRk7So>